Master Test Plan

Projekt 1

Kunststoff4All

Version 0.3.0

Änderungshistorie

Version	Kommentar	Datum	Name
0.1.0	Ersterstellung	09.04.2025	NB
0.2.0	Erweiterung und Anpassung	11.04.2025	NB
0.3.0	Erweiterung und Anpassung	28.04.2025	NB
0.4.0	Erweiterung und Anpassung	02.05.2025	NB
0.5.0	Erweiterung und Anpassung	16.05.2025	NB
0.6.0	Erweiterung und Anpassung	23.06.2025	NB
0.7.0	Erweiterung	25.06.2025	NB

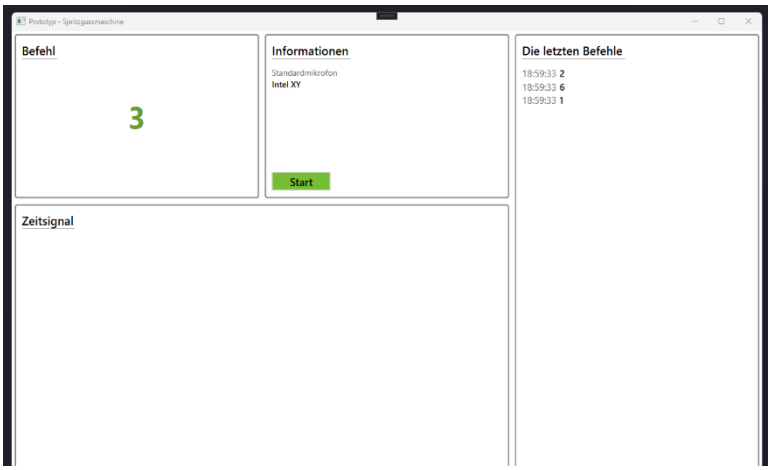
Inhaltsverzeichnis

1 Projekt 5
1.1 Kurzbeschreibung 5
1.2 Ziele 5
1.3 Referenzen 5
1.4 Verantwortlichkeiten 6
2 Relevante Standards 7
2.1 Nationale und internationale Standards 7
3 Testziele 8
3.1 Kurzbeschreibung 8
4 Teststrategie 8
4.1 Grundsätzliche Vorgehensweise 8
4.2 Qualitätskriterien 8
4.2.1 Funktionalität ..... **Fehler! Textmarke nicht definiert.**
4.2.2 Zuverlässigkeit ..... **Fehler! Textmarke nicht definiert.**
4.2.3 Benutzbarkeit ..... **Fehler! Textmarke nicht definiert.**
4.2.4 Effizienz ..... **Fehler! Textmarke nicht definiert.**
4.2.5 Sicherheit (Security) ..... **Fehler! Textmarke nicht definiert.**
4.2.6 Kompatibilität ..... **Fehler! Textmarke nicht definiert.**
4.2.7 Wartbarkeit ..... **Fehler! Textmarke nicht definiert.**
4.2.8 Portabilität ..... **Fehler! Textmarke nicht definiert.**
4.3 Risiken 10
4.4 Teststufen 11
4.4.1 Model Test (MT) 11
4.4.2 Model in the Loop (MiL) 11
4.4.3 Rapid Prototyping (RP) 11
4.4.4 Software Unit – Host (SiL-H) 11
4.4.5 Software Unit – Target (SiL-T) 12
4.4.6 Software Integration – Target (SWI-H) 12
4.4.7 Software Integration – Target (SWI-T) 12
4.4.8 HW/SW Integration (HiL) 12
4.4.9 System Integration (SI) 12
4.4.10 System Test (ST) 12
4.4.11 Pre-Production (PP) 12

4.4.12 Feldtest (FT) 13
4.4.13 Akzeptanztest (AT) 13
4.5 Teststrategiematrix 14
5 Grober Zeitplan 15

1 Projekt

1.1 Kurzbeschreibung



Es soll eine Sprachsteuerung für die Spritzgussmaschine entwickelt werden, welche anhand der Sprachbefehle 1-9 ein Zeitsignal erstellt und dieses auf der Benutzeroberfläche darstellt und ausführt.

Die Lieferung umfasst eine Sprachauswertung der besagten Befehle mit entsprechender Bedienoberfläche für Personal.

1.2 Ziele

- **Umsetzbarkeit nachweisen**

  Durch den funktionfähigen Prototyp soll gezeigt werden, dass die Sprachsteuerung trotz Umgebungsgeräuschen, umsetzbar ist.

- **Simple und intuitive Benutzeroberfläche**

  Die GUI soll benutzerfreundlich und übersichtlich konstruiert sein, um eine Bedienung ohne Einarbeitung zu ermöglichen.

- **Wiederverwendbarkeit und Erweiterbarkeitdurch Modularität**

  Ein modularer Prototyp soll für spätere Weiterentwicklungen, aber auch für die Integration im finalen Produkt verwendbar sein.

- **Funktionstüchtige & Präsentationsreife Anwendung**

  Die Anwendung soll trotz Geräuschkulisse, zuverlässig die richtigen Befehle erkennen und ausführen. Genauso soll die Anwendung korrekte Historien, Mikrofondaten und Zeitsignale ausgeben.

- **Arbeitsaufteilung durch Modularität**

  Durch die Aufteilung der Aufgaben und das herunterbrechen in Module, soll das Entwicklerteam eine gleichmäßige Arbeitsaufteilung umsetzen.

1.3 Referenzen

- *Anforderungsdokument*
- *Lastenheft*
- *Pflichtenheft*
- *Storyboards*
- *Systemarchitektur*
- *Softwarearchitektur*

- *Verweise auf Projekte in Application Lifecycle Tools (Jira, Polarion, …)*

1.4 Verantwortlichkeiten

Auftraggeber: Kunststoff4All

Projektleiter: Anton Armin

Entwicklungsleiter: Benz Jan

Testmanager: Bazaj Donart

Projektkoordinatorin: Nadine Bichler

2 Relevante Standards

2.1 Nationale und internationale Standards

- *ISO/IEC 25000:2014 Systems and software engineerng – Systems and software Quality Requirements and Evaluation (SQuaRE) – Guide to SQuaRE*
- *ISO/IEC 25010:2023 Systems and software engineerng – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*
- *ISO/IEC/IEEE 29119-1:2022 Software testing – Part 1: General concepts*
- *ISO/IEC/IEEE 29119-2:2021 Software testing – Part 2: Test processes*
- *ISO/IEC/IEEE 29119-3:2021 Software testing – Part 3: Test documentation*
- *ISO/IEC/IEEE 29119-4:2021 Software testing – Part 4: Test techniques*
- *ISO/IEC/IEEE 29119-6:2021 Software testing – Part 6: Guidelines for the use of ISO/IEC/IEEE 29119 (all parts) in agile projects*
- *MISRA-C:2012*

3 Testziele

3.1 Kurzbeschreibung

Es soll eine Sprachsteuerung für die Spritzgussmaschine entwickelt werden, welche anhand der Sprachbefehle 1-9 ein Zeitsignal erstellt und dieses auf der Benutzeroberfläche darstellt.

Die Lieferung umfasst eine Sprachauswertung der besagten Befehle mit entsprechender Bedienoberfläche für Personal.

Die Testziele des Projekts „Sprachsteuerung für die Spritzgussmaschine“ umfassen:

- Sicherstellung der funktionalen Korrektheit der Spracherkennung für die Ziffern 1–9.
- Richtige Visualisierung des erzeugten Zeitsignals in der Benutzeroberfläche.
- Überprüfung der Systemreaktion bei Umgebungsgeräuschen (Robustheit).
- Verifikation der Protokollierung (Historie, Zeitsignal, Mikrofondaten).
- Überprüfung der Benutzerfreundlichkeit (GUI, Systemfeedback).
- Validierung der Systemintegration (Hardware, Mikrofon, GUI, Logik).

4 Teststrategie

4.1 Grundsätzliche Vorgehensweise.

Die Teststrategie folgt einer iterativen Vorgehensweise auf Basis der Testpyramide:

- Unit Tests (Spritzgusslogik, Einzelkomponenten)
- Integrationstests (Verknüpfung von Mikrofon, Spracherkennung, GUI)
- Systemtests (Gesamtsystem, inkl. Hardware & Sprache)
- Abnahmetests (Demo mit realen Nutzern)

Testfälle werden vor der Implementierung spezifiziert, bei Änderungen aktualisiert und automatisiert, wo möglich.

4.2 Qualitätskriterien

Kriterium	Testansatz	Relevanz
Funktionalität	- Modul: Spracherkennung - Submodule: Mikrofonaufnahme, MFCC-Extraktion, Vergleich mit Samples - Test: Sprachbefehl korrekt erkannt?	Hoch
Zuverlässigkeit	- Reaktion bei Rauschen, falscher Eingabe - Konsistente Ergebnisse bei mehrfacher Eingabe - Messung der Fehlerrate	Hoch

Benutzbarkeit	- GUI verständlich? - Sprachfeedback? - Ergonomietest mit Testpersonen	Mittel
Effizienz	- Verarbeitung innerhalb von < 1s - Ressourcenverbrauch gering - CPU-Last beim Live-Recording	Mittel
Sicherheit	- Unzulässige Befehle dürfen keine Wirkung zeigen - Systemzugriffe nur intern erlaubt - Keine Verarbeitung bei unvollständiger Spracheingabe	Hoch
Kompatibilität	- Abspielen von .wav-Files auch von Dritten	Mittel
Wartbarkeit	- Klare Modulschnittstellen - Logging von Erkennungsergebnissen - saubere Kapselung der Funktionen	Mittel
Portabilität	- Verlagerung in Embedded-System perspektivisch vorgesehen	Niedrig

Konflikt:

- Hohe Zuverlässigkeit erfordert ggf. stärkere Filter → das kann die Effizienz mindern.
- Einfache Wartbarkeit (modular) kann die Performance verschlechtern.

Priorisierung

Kriterium	**Relevanz**
Funktionalität	1
Zuverlässigkeit	2
Benutzbarkeit	3
Effizienz	4
Sicherheit	5
Kompatibilität	6
Wartbarkeit	7

Portabilität	8

4.3 Risiken

Risikoklassen sind zu allen Qualitätskriterien festlegen (tabellarisch). Hohes Risko A, mittleres Risiko B, niedriges Risiko C, kein Risiko -.

Kapitel könnte auch im vorherigen Kapitel bei den Qualitätskriterien genannt werden. Aufgrund der Wichtigkeit ist aber ein eigenes Kapitel zu empfehlen.

Qualitätskriterien	Risiko
Funktionalität	
• Modul A	A
• Modul B	B
• Modul C	-
Zuverlässigkeit	-
Benutzbarkeit	B
Effizienz	-
Sicherheit/Security	A
Kompatibilität	-
Wartbarkeit	-
Portierbarkeit	-

Kriterium	**Risiko**	**Begründung**
Funktionalität	A	Fehlerhafte Spracherkennung kann Maschinenfunktionen fälschlich auslösen.
Zuverlässigkeit	A	Unklare oder wechselnde Erkennung gefährdet die Bedienbarkeit.
Benutzbarkeit	B	Verwirrende GUI kann Bedienfehler verursachen, z. B. falsche Zifferneingabe.
Effizienz	B	Verzögerte Erkennung führt zu Frust oder Sicherheitsrisiken.
Sicherheit	A	System darf keine gefährlichen Zustände bei Fehlbedienung erreichen.
Kompatibilität	C	Einschränkungen bei Hardware beeinträchtigen Portabilität, nicht die Funktion.
Wartbarkeit	B	Schlechte Kapselung kann Fehlerbehebung behindern.
Portabilität	C	Spätere Systemmigration unwahrscheinlich, geringes Risiko.

4.4 Teststufen

4.4.1 Model Test (MT)

Ziel: Test der Signalverarbeitung (z.B. MFCC) unabhängig von Spracheingabe

Testart: Unit-Test mit predefinierten Testdaten

Werkzeug: MSTest, Unit-Tests in C#

4.4.2 Model in the Loop (MiL)

Ziel: Vergleich des Audiosignals mit dem Modell (MFCC-Vergleich)

Testart: Blackbox-Analyse gegen Referenzdaten

Werkzeug: Python + gespeicherte CSV-Vektoren

4.4.3 Rapid Prototyping (RP)

Ziel: Schneller Gesamttest der Spracheingabe bis GUI-Ausgabe

Testart: Explorative Tests während der Entwicklung

4.4.4 Software Unit – Host (SiL-H)

Ziel: Einzeltests von audioConvert, SignalIdentifier, Microphone

Was wird getestet:

Da alle zentralen Funktionen des Systems (z. B. Audioverarbeitung, MFCC-Vergleich, GUI-Reaktion, History-Management) auf dem Host-PC in C# oder Python implementiert sind, ist eine isolierte Testung auf Modulebene notwendig und technisch vollständig möglich.

- *SignalIdentifier (Modul für Spracherkennung und Ereignisauslösung)*
- *audioConvert (Modul für WAV-Speicherung und CSV-Handling)*
- *Microphone (Logik zur Audioerfassung, Voice-Aktivitäts-Erkennung)*
- *DigitRecognizer (Python-Logik für MFCC-Vergleich)*
- *Fehlersituationen, z. B.:*
    - *leeres PCM-Signal*
    - *ungültiger Dateiname*
    - *beschädigte Sample-Dateien*
    - *keine Übereinstimmung erkannt*

Verweis auf Dokumente:

- Pflichtenheft
- Systemtest

Testart:

- White-Box-Testverfahren
    - Äquivalenzklassen
    - Grenzwertanalyse
    - Kontrollflussorientierte Tests (z. B. if (similarity > threshold))

- Fehlerszenarien / Robustheitstests
    - Null-Referenzen
    - nicht vorhandene Dateien
    - fehlerhafte Dateiinhalte
- Regressionstests
    - Tests werden nach jeder Änderung automatisiert neu ausgeführt

Tools: Unit-Tests in C#, MSTest, Python

4.4.4.1 Testeingangskriterien

4.4.4.2 Testabbruchskriterien

4.4.4.3 Testwiederaufnahmekriterien

4.4.4.4 Testendekriterien

- Decision Coverage 100%
- Statische Code Analyse ohne Warnung
- Code Review bestanden
- No Warnings bei -WALL
- Cognitve Complexity jeder Funktion < 10

Eventuell hier den Testreport aufzählen

4.4.5 Software Unit – Target (SiL-T)

- Nicht durchgeführt – Target-System nicht vorgesehen

4.4.6 Software Integration – Target (SWI-H)

- Ziel : Zusammenspiel von Signalverarbeitung, Python-Backend und C#-Frontend
- Testart: Integrationstest

4.4.7 Software Integration – Target (SWI-T)

- Nicht druchgeführt – kein Embedded-System

4.4.8 HW/SW Integration (HiL)

- Ziel: Mikrofon + Lautsprecher + GUI im Zusammenspiel
- Testart: Hardware-in-the-Loop mit Live-Spracheingabe

4.4.9 System Integration (SI)

- Ziel: Vollständige Sprachsteuerung inkl. Reaktion der Oberfläche
- Testart: Blackbox-Gesamttest

4.4.10 System Test (ST)

- Ziel: Validierung aller Anforderungen unter realen Bedingungen
- Kriterien: Sprachverständlichkeit, GUI-Reaktion, Feedback korrekt

4.4.11 Pre-Production (PP)

- Ziel: Test mit internen Pilotnutzern
- Anwendung: Übergabe an finales UI-Team

4.4.12 Feldtest (FT)

- Ziel: Einsatz in einer echten Umgebung mit Bedienpersonal
- Ergebnis: Rückmeldung zur Erkennung unter Umgebungsgeräuschen

4.4.13 Akzeptanztest (AT)

- Ziel: Präsentation & Übergabe an Auftraggeber
- Erfolgsbedingung: 100% korrekt erkannte Sprachbefehle

4.5 Teststrategiematrix

			Teststufen												
Qualitätskriterien	Risiko	Review	MT	MiL	RP	SIL-H	SIL-T	SWI-H	SWI-T	HiL	SI	ST	PP	FT	AT
Funktionalität	A	ja	●●●	●●●	●	●●●		●●		●●●	●●●	●●●	●●	●●	●●●
• Modul A	A	ja													
• Modul B	B	ja													
• Modul C	-	ja													
Zuverlässigkeit	A	ja	●●	●●●	●	●		●●		●●●	●●	●●●	●	●●	●●●
Benutzbarkeit	B	ja	●●●		●●			●		●●●	●●	●●●	●●●	●●	●●●
Effizienz	B	nein	●	●	●	●●		●		●●	●	●●		●	●●
Sicherheit/Security	A	ja	●●					●		●●●	●●	●●●	●	●●	●●●
Kompatibilität	C	nein			●					●●	●	●●	●	●●	●
Wartbarkeit	B	nein				●●●		●				●			
Portierbarkeit	C	nein										●	●		

Legende:

Intensität des Testens:	niedrig: ●	mittel: ●●	hoch: ●●●	
Risikoklassen:	niedrig: C	mittel: B	hoch: A	Kein Risiko: -
Implizites Testen:	I			

5 Grober Zeitplan

Phase	**Datum**
Idee/Kickoff	15.04.2025
Systemanforderungen	20.4.2025
Systementwurf	04.05.2025
High-Level-Design	25.05.2025
Low-Level-Design	01.06.20205
Implementierung	27.06.2025
Tests & Reports	27.06.2025
Deployment	27.06.2025
DevOps, Testmanagement, Projektmanagement	27.06.2025